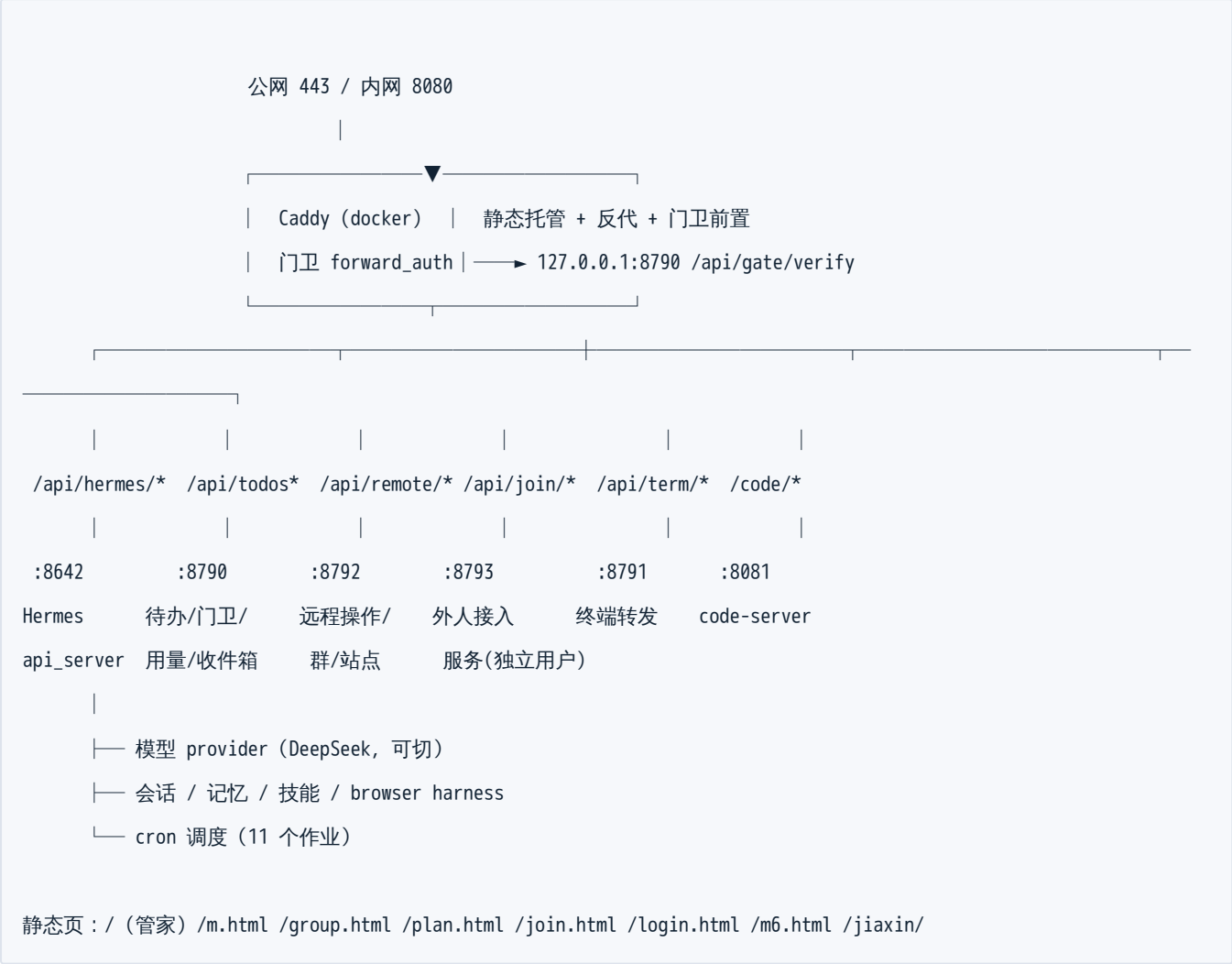
咕嘎系统 · 设计文档

版本：2026-10-17　|　维护：咕嘎一号　|　对应文档：《咕嘎系统 · 功能文档》

一、总体架构

一句话：一个反代（Caddy）打头，后面挂一堆各管一摊的小服务，核心是 Hermes Agent 的 API 网关；前端是纯静态页面。



三条设计原则：

1. 能不加组件就不加：能用 `http.server` 写的就写，能挂在已有反代路径下的就挂，绝不为此改反代配置。
2. 每个服务只做一件事、只绑回环：除 Caddy 与两个终端端口，所有后端都只监听 `127.0.0.1`，公网进不来。
3. 可还原：动到配置、门卫、接入、前端，一律先备份再配一键还原脚本。

二、物理与系统底座

项	实际
服务器	腾讯云轻量应用服务器（Lighthouse，不是 CVM，开关端口在「防火墙」页签），广州，4 vCPU / 4 GB / 40 GB
系统	Ubuntu 24.04，`sudo` 免密，`ufw` 只放 22/80/443
运行时	Hermes Agent v0.21.6（git 安装），模型 `deepseek-flash`（provider: deepseek）
容器	Docker：`caddy:2`（:80/:443/:8080）、`gitea/gitea:1`（127.0.0.1:3000）
常驻服务	systemd 8 个：`hermes-todo-api`、`guga-remote-api`、`guga-join-api`、`guga-term-api`、`guga-jiaxin-api`、`guga-group-relay`、`guga6-watch`、`ttyd-hermes`（另两个 ttyd 与 code-server 各自进程）
内存策略	开了 swap；内存常年紧张（这是「重活外派四号」的直接原因）

端口一览（实测监听表）

端口	服务
22	sshd（含反向隧道绑定段 22022–22039）
80 / 443	Caddy 公网站点
8080	Caddy 私网站点（免密后门，只走隧道）
2019	Caddy 管理口（回环）
3000	Gitea（docker）
7681 / 7682 / 7683	ttyd：管家终端 / 编码终端 / 工作区 shell
8081	code-server（VS Code 网页版）
8642	Hermes api_server（主入口）
8790	`todo_api.py`：待办 / 门卫校验 / 用量 / 收件箱 / 上传 / 资源面板
8791	`term-api.py`：终端会话管理（94 行）
8792	`remote-api.py`：远程机器 / 群 / 站点（718 行）
8793	`join_api.py`：外人接入（跑在 `/opt/guga-join`，系统账户 `gugajoin`）
8794	`jiaxin_api.py`：夹心页数据（132 行）
9333	常驻 Chrome 的调试口（WorkBuddy / 抢码用）

三、模块：反代与门卫

技术栈：Caddy 2（单文件 `Caddyfile`，以 bind mount 挂在容器里）。

实现要点

- 公共路由抽成一个片段（snippet），私网站点 `:8080` 与公网站点（域名 + www）各自 `import`，一份定义两处生效。
- 路径与后端：`handle_path /api/remote/*` 会剥掉前缀再转发（后端因此拿到的就是 `/group`、`/sites`），`handle /api/todos*` 不剥。这个差别决定后端路由怎么写，混用必 404。
- 静态站与 API 混在一起：`/` 指向 bind mount 的 `hub/` 目录，改文件即时生效，不需要重启任何服务。
- 门卫（2026-10-13 换掉浏览器原生 basic_auth）：
- 整站 `forward_auth → 127.0.0.1:8790/api/gate/verify`；
- 没通过时由门卫自己回 302 到 `/login.html?next=…`（Caddy 的 `handle_errors` 抓不住 forward_auth 传回来的非 2xx，这是关键写法）；
- 登录 `POST /api/gate/login` 换 HMAC 签名的 cookie `guga_gate`；账密只存 pbkdf2 哈希（`~/.hermes/secrets/`）；
- 两个账号：`mqd`（cookie 10 年）、`guga`（会话 cookie，关浏览器即失效）；页面 `autocomplete=off`，密码框每击键重掷随机字符，真密码只存在 JS 变量里。
- 公网白名单式免密：只给 `join` 那几条接口开洞，兜底 `respond 404`；用 `uri strip_prefix /api` 对齐后端路径；上游要管理密钥的路径只在私网站点，由 Caddy 注入管理头。

关键取舍 / 坑

- 单文件 bind mount 改了 `caddy reload` 常常静默无效（容器里还是旧 inode，回一句 `config is unchanged`）→ 必须 `docker restart caddy`。这是本项目最贵的一条教训。
- 公网可达性只认外部探测（本机 curl 自己的公网 IP 会 hairpin 超时），否则会把「云防火墙没放 443」误判成「服务挂了」。

四、模块：前端（hub 是纯静态页）

技术栈：原生 HTML + CSS + JavaScript，无框架、无构建、无 npm。总计约 6800 行：

页面	行数	角色
`index.html`	2925	桌面主页面：四 tab、聊天、待办面板、远程连接、站点页
`m.html`	1106	手机版管家（会话抽屉 + 抽屉式待办/站点/群）
`m6.html`	1090	六号手机版（由 `make-m6.py` 从 `m.html` 生成，不手改）
`plan.html`	717	人生规划页（段位 / 阶段 / 今日任务）
`join.html`	457	外人接入页
`group.html`	375	咕嘎群
`login.html`	136	门卫登录页

实现要点

- 流式聊天：`fetch('/api/hermes/v1/runs')` 拿到 run id，再 `GET /v1/runs/<id>/events`，自己解析 SSE（`readSSE/streamRun`），带 `seq / last_seq` 续传——网断了重连接着读，不重跑。
- 渲染指令：正文里单行 `::options` → 其后的「1. …」行渲染成按钮（点了就发一条用户消息）；单行 `::sysinfo` → 拉 `/api/sysinfo` 画资源面板；单行 `•` → 底部蓝点；`http…` → `<a target=_blank>`。
- 面板搬移：切到「远程连接」时把管家那块聊天面板整块搬进该视图、切走再搬回（`moveChatPane / switchChatPane`）。有副作用的写法：某些 tab 的 else 分支会清空聊天，所以新页面必须是「安全类型」（`data-tab` 不是 `butler` / `local`）。
- 附件：回形针 + 拖拽 + 粘贴三条路，前端把文件 POST 到 `/api/inbox/upload`（raw 字节当 body，文件名放 `X-File-Name` 头，不用 multipart），成功后把「【附件】路径（原名，大小）」拼在消息开头，管家据此读文件。
- 手机版生成：`make-m6.py` 靠精确锚点改写 `m.html` 生成 `m6.html`（六号专属：API 基址换成远程分身、加连接信息卡）。锚点被改动就中断——改 `m.html` 必须同步改脚本。

关键取舍

- 不引框架的代价是每加一个页面都有重复代码，好处是零构建、零依赖、改完刷新即生效，在一台 4G 内存的机器上很值。
- 验证方式固定：改完用无头 Chromium `--dump-dom` 看真实渲染结果；要测点击用 CDP 的 `Input.dispatchMouseEvent`（`element.click()` 不是用户手势，`target=_blank` 会被拦）。

五、模块：Hermes 内核接入层

技术栈：Hermes Agent（Python），`api_server` 网关 `127.0.0.1:8642`；模型 provider 可切（当前 DeepSeek）。

实现要点

- 无状态会话：前端只带会话 id，网关不持有 chat 上下文，历史在内核侧按会话存。

- 持久运行：`POST /v1/runs` 起一次运行 → `GET /v1/runs/<id>/events` 读事件流（带序号可续传）。hub 的聊天、群信使、搬运工全部走这条，而不是 chat/completions。
- 人格注入：起运行时带 `instructions`（临时系统提示，不落进历史）。远程机器会话注入「你就是咕嘎N号 + 它的专属记忆」，六号会话注入身份提示并把会话绑到那台 guest。
- 多 profile 复用同一网关：管家 = `default`（完整记忆）；远程分身 = `profiles/remote`（空记忆、`skills/` 软链共用、自己一份 SOUL）。前端按面板切基址 `/api/hermes` 与 `/api/hermes/p/remote`，于是同一个界面里两套记忆互不可见。新建 profile 必须重启网关才进多路复用。
- cron 调度器在内核里：作业定义一份 `jobs.json`，支持两种模式——agent 任务（跑模型，用于需要判断的早报晚报）与 no-agent 脚本任务（脚本 stdout 直接投递，零模型成本）。
- 投递策略：全部 `deliver=local`，输出落 `cron/output/<job_id>/*.md`，由 `/api/inbox` 在界面打开时领取。少数作业走 `--deliver feishu`（在四号那台机器上）。
- 消息出口：本机 `hermes send` 支持纯文本 / 文件 / `MEDIA:` 附件，但本机没接消息平台，实际出口在四号。

关键取舍

- `MEDIA:` 贴图只在部分端点生效（chat/completions / responses），走 `/v1/runs` 的 hub 不认 → 看图一律「拷进静态目录 + 给链接」。

六、模块：待办服务（`:8790`）

技术栈：Python 标准库 `ThreadingHTTPServer`（`todo_api.py`，915 行）+ 命令行工具 `todo.py`（616 行）+ Markdown 单文件存储。没有数据库、没有框架。

实现要点

- 真源是一份 Markdown（按日期分节，`- [ ]` / `- [x]`，尾部可带 `@日期 时刻`）。所有读写都过 `todo.py`，前端与管家调同一套命令，人肉改文件只作为最后手段。
- 多线程而非单线程：一条会阻塞的调用（ssh、外网探测）不能卡住整个面板轮询。
- HTTP 接口面（同一进程同一端口，靠路径分发）：

路径	作用
`/api/todos`	三档清单（today / tomorrow / long），顺带下发段位数据
`/api/inbox`、`/api/inbox/upload`、`/api/inbox/uploads`	收件箱领取、附件上传、最近上传列表
`/api/sysinfo`	磁盘 / 内存 / 交换
`/api/usage`	当日 token 与花费（读用量记录）
`/api/gate/verify`、`/api/gate/login`	门卫（forward_auth 的目标）

- 每日播种：固定项（吃药 22:00 等）在「当天首次读取」时插到顶部，状态记一份播种台账（每日期一次），所以当天删掉不会自己回来。
- 段位联动：规划页的段位/阶段数据挂在 `/api/todos` 的响应里，带 60 秒缓存，避免每次刷新都重算四份数据源。
- 上传：raw body + `X-File-Name`（percent-encoded），64 MB 上限，落 `hub/uploads/<日期>/`；因为 hub 目录本身是静态托管目录，缩略图直接能当静态资源用。

为什么这么写：这台机器跑不起一堆服务，把「同样要吃 todo.py 的接口」合并进一个进程，端口与 systemd 单元都省了；代价是这个文件变成了枢纽（915 行），改动要格外小心。

七、模块：远程操作（`:8792`）

技术栈：Python 标准库 `ThreadingHTTPServer`（`remote-api.py`，718 行）；传输层用 ssh 别名 + 反向隧道（不装 agent、目标机零常驻依赖）。

实现要点

- 自有机器登记：`hosts.json` 是真源 → 生成器 `gen-config.py` 写出 ssh 配置文件 → `ssh <别名>` 直接可用。反向隧道把目标机 22 挂到服务器回环 22022–22039 段，所以「那台机器在 NAT 后面」不是问题。
- 确认门 `classify()`：含 shell 元字符一律判危险；否则查只读白名单；default-deny。危险/写操作必须先批准，只读 guest 模式下直接 403。每条命令先写目标机本地审计日志再执行，服务器侧留 JSONL 审计。
- 多会话模型：每台机器映射到一段「远程 · 机器名」会话 + 一份专属记忆文件；同名会话标题不允许重复，所以第 2 段起标题加序号。
- 群与站点同一进程：
- 群：`group.json` 为唯一真源，原子写（tmp + `os.replace`）；`id` 单调递增、删除不回退（前端靠 `id > lastId` 增量取）；条数上限与单次返回上限分开；`who` 必须在校验名单里。
- 站点页：`sites.json`；URL 只收 `http(s)://` 或 `/` 开头（挡 `javascript:` / `data:` / `//evil`），颜色只收 `#RRGGBB`，所有文字走 `textContent`；预览页文件被自动发现成只读条目。
- 为什么同一个文件：群与站点都要读「机器会话 / 局域网身份」这批数据，拆成两个服务就要共享状态；合并在一个进程里可用内存锁。代价是 `remote-api.py` 成为第二个枢纽，改动只许管家做。

八、模块：外人接入服务（`:8793`）

技术栈：Python 标准库 + sshd 的 `AuthorizedKeysCommand` 机制 + tmpfs。运行在 `/opt/guga-join`，以系统账户 `gugajoin` 跑（读不到用户工作区），由 Caddy 注入管理头访问管理接口。

核心机制

1. 对方打开接入页 → `POST /connect`（要密码，除非带邀请令牌）签发一次性 token（10 分钟）。

2. 页面给出命令 → 目标机执行 → 向服务器 `claim` → 服务器生成当次专用公钥，写进 `/run/guga-join/authorized_keys`（内存盘）。
3. sshd 侧 `Match User tunnel` + `AuthorizedKeysCommand` 让受限用户 `tunnel` 只认这份内存盘里的钥匙——不用 sudo、不改服务硬化、机器一重启就清空（天然无痕）。
4. 目标机用这把钥匙建反向隧道（端口段 22022–22039），页面转「已连接」；之后命令走 `/exec` + `/approve`。
5. 租约：接入页每 20 秒心跳续租，`LEASE=130` 秒；服务端一个清扫线程每 10 秒扫一遍，过期即撤销。所以关掉页面约 2 分钟自动断开。
6. 真正拆隧道的是目标机：服务器杀不掉受限账户下的进程，所以目标机的保活循环每 15 秒问一次「还活着吗」，服务器明确说「没了」才自己拆（网络抖动不算）。

关键取舍

- 令牌 = 短命 bearer 凭证：拿到命令的人能在 10 分钟内、一次、免密连进来。这是刻意的取舍（换成多次校验会让「手机开页面、电脑跑命令」这种真实用法变得难用），窗口用 TTL 与一次性收紧。
- 页面永不预签令牌：早先在页面加载时就预签，开几次名额就挤满（全 429）；改成用户点「同意」才签，不同入口名额分开算。
- 执行策略死锁的兼容：Windows 目标机若被组策略锁了脚本执行，任何「加载 .ps1」的做法都会失败 → 接入脚本改成把代码内联成 base64 命令（`-EncodedCommand`），并优先用系统能力包装 OpenSSH。
- 复制粘贴的坑：给主人的命令里 URL 绝不能带 `&`（PowerShell 视为保留符），参数一律塞路径段，于是整条命令可以零引号；页面还带版本号自检，缓存里的旧页面会被横幅提示强刷。

九、模块：群信使（`guga-group-relay`）

技术栈：Python 守护进程（`group_relay.py`，599 行）+ `group.json` + Hermes runs 通道。

实现要点

- 3 秒轮询（不是长连接）：读取群文件，判断要不要叫醒人。轮询比推送简单，且天然容忍重启。
- 回话条件三选三才回：主人的消息存在；主人最后一条之后没有任何咕嘎发言；这条还没答过（进程内记 `done_id`）。于是「连发多条只回一次、自己发言不触发下一轮、重启不重复回」三件事同时满足，不用额外锁。
- 路由：消息带 `to`（页面点头像）> 话里叫名字 > 在线成员全回。斥候另有一条关键词路由（热搜/热点/新闻/大事 + 查/看/盯/搜）。
- 执行：`POST /v1/runs` 带 `instructions`（「你现在在群里说话，回话要短、不要渲染指令、别写旁白」），流式读到完成，取文后 `tidy()` 剥掉结尾蓝点与 `::xxx` 指令行，再以「本人 / 回话」身份贴回群；回话期间置 busy，群里显示「X 正在想…」。
- 日志轮转：自己控制大小（超 400 KB 只留尾部），因为这台机器盘不大。

十、模块：搬运工与浏览器自动化

技术栈：Python + browser harness（CDP）+ 常驻 Chrome 固定档案。

为什么需要常驻浏览器：默认浏览器会话每次重启都是空档案（临时目录还会被定期清理），登录态一天就蒸发。解法是起一台常驻 Chrome（固定 `--user-data-dir` + 固定调试口 9333），让 harness 用 `BU_CDP_URL` 接管它、用独立 daemon 名避免跟默认会话打架。登录态从此跨重启保留；凭证另存一份到非缓存路径，做保鲜探针，到期前在晚报里提醒。

两条搬运线

- 四号（LightVela）：`agent_carry.py` 调它的 debug/exec 接口，在它实例里跑一轮对话（自己的记忆、自己的会话），原话取回贴群标代发；失败会重开会话重试一次。
- 三号（WorkBuddy）：`workbuddy_ask.py` 用它自己的常驻浏览器在它自己的任务会话里打字 → 轮询 DOM 判断「已完成」→ 取正文剥掉页面标签 → 贴群。代价：每答一句吃它平台一次调用额度，所以排最后说。

取舍：浏览器自动化天生脆（软件改版就失效），但这是唯一能用上那两个平台的通道；因此把「判断跑完的标志」写成可替换的锚点，并单独记了一份踩坑清单。

十一、模块：跨机消息桥（本机 → 四号 → 飞书）

技术栈：Python 脚本 + base64 过桥 + 四号的 no-agent 脚本作业。

数据流

```
本机 cron 生成早/晚报（落 cron/output/…md）
   |  brief_to_lv.py 取正文 → 滤掉 ::xxx 与结尾蓝点
   |  base64 经 exec 接口写进四号 /home/agentuser/.hermes/brief/<日期>-{morning,evening}.txt
   ▼
四号两个 no-agent 作业（08:06 / 22:09, --deliver feishu）
   |  脚本只做一件事：cat 那个文件
   ▼
主人飞书（一字不改）
```

要点与坑

- 时间顺序是关键：本机 08:00 / 22:00 先跑完才落盘，所以推送窗设成 08:04–08:34（每 5 分钟一档、共 7 次重试），四号 08:06 才发。
- 去重：同一天同一份内容按哈希只推一次；状态存本地 JSON。
- 「跑完了」≠「送到了」：任务列表里的 ok 只说明脚本退出码正常，真正的投递结果要查四号那边的执行记录（`delivery_outcome='delivered'`）。

- 单向：四号没有回到本机的通道（本机没公网入口），所以联动永远是「我主动调它」。

十二、模块：定时调度与夜间自动化

技术栈：内核 cron 引擎 + shell / Python 脚本（真身放各自咕嘎的 `scripts/`，`~/.hermes/scripts/` 只放几行启动器——cron 会把软链判成越界直接拦掉，所以启动器必须是真文件并 `exec` 真身）。

11 个作业的分工

类别	作业
判断型（agent）	早报、晚报、周复盘、热点日报
脚本型（no-agent）	晚安、到点提醒、夜间扫描/工程/自检、简报过桥、浏览器清理、交换空间巡逻、任务完成通知
频次	每分钟（完成通知）/ 5 分钟（到点提醒）/ 每半小时（浏览器清理）/ 每日 / 每周六

夜间三件事的分层：01:00 找料写想法池 → 晚报呈给主人 → 主人在批准清单里点头 → 02:00 只做清单里的 → 03:00 自检与清旧备份。「找料」与「动手」被强制拆成两个作业、两个文件，这样「不自行挑活」是结构性的，不靠自觉。

十三、模块：人生规划页与段位判定

技术栈：Python 脚本（`plan_ranks.py`）+ 静态页 + 复用 `/api/todos` 下发数据。

实现要点

- 判定与呈现分离：脚本 `compute()` 算段位、`write()` 落盘；页面只负责渲染，不参与判定。
- 数据源全是只读：待办勾选（`[x]` 才算事实）、上床时间记录、投递记录表、手动置位 JSON。算不出来的条件（offer、体能有可指变化、公开发布、起床稳定）走手动置位。
- 只升不降：当前段位 = 1 + 从第一条起连续达成的目标数。
- 两处常量必须同步：阶段边界在两个文件里各有一份（脚本 + 出图脚本），改一处等于埋一个「页面和卡片不一致」的雷。
- 缓存：段位挂在 `/api/todos` 里带 60 秒 TTL，所以「改了手动置位文件却看不到变化」＝缓存还热，不是逻辑没生效。

十四、模块：巡检与瘦身

脚本	机制
`night-watch.sh`	只读自检（服务在不在、磁盘、日志）+ 删 7 天前备份
`swap-patrol.py`	交换空间占用 ≥70% 才出声，顺手清掉超过阈值的浏览器会话
`browser-daemon-reaper.sh`	每半小时清残留的浏览器 daemon
`guga6_watch.py`（常驻服务）	盯接入流水，落台账与事件流

设计取向：安静。不达标不出声，出声必带可执行动作（清什么、清多少），避免「天天报警等于没报警」。

十五、模块：记忆与技能

技术栈：Markdown 文件 + 内核的常驻记忆注入 + 技能按需加载（`SKILL.md` 带 frontmatter）。

分层实现

层	实现	何时进入上下文
常驻记忆	一份 `MEMORY.md` + 一份 `USER.md`，有硬字符预算	每一轮都注入
技能	每个技能一个目录 + `SKILL.md`（描述即触发器）	按任务按需加载
知识分区	`knowledge/procedures.md`（流程与坑）、`knowledge/projects/<项目>.md`（一项目一份）、价值观/饮食/想法各自成文	需要时读
待办/提案	单文件 + 脚本	需要时读

原则：任务里学到的东西写进技能或项目文档，不往常驻记忆里堆——常驻记忆只放「每轮都必须知道」的身份与操控规矩。这一条是这套系统能长期不退化的关键。

十六、四条典型数据流

① 发一条聊天消息 前端 → `POST /api/hermes/v1/runs`（可能带 `instructions`）→ 内核跑工具循环（模型 = DeepSeek）→ 前端读 `/events` 流式渲染 → 完事按渲染指令出按钮/面板/蓝点。

② 一次早报到手机 08:00 内核 agent 任务生成正文落盘 → 08:04 起过桥脚本推到四号 → 08:06 四号脚本 `cat` 后投飞书 → 主人手机看到的就是本机原文。

③ 外人接入一台电脑 打开 join 页 → 输密码 → 签发一次性 token → 目标机跑命令 → `claim` 生成专用公钥写内存盘 → sshd 认钥匙 → 反向隧道建立 → 页面「已连接」→ 命令先审计后执行 → 关页面 2 分钟租约到期 → 目标机自查发现会话没了 → 自己杀隧道、删公钥、注销计划任务。

④ 群里叫一个人 主人发言（或点头像点名）→ 信使 3 秒内看到「主人的新消息且之后无人发言」→ 按路由决定叫谁 → 起一次运行、注入群内身份 → 收文、剥渲染指令 → 以「本人/代发」贴回群、清 busy。

十七、可靠性与安全设计

- 备份与还原：每次动配置/前端/门卫/接入，先 `cp` 带时间戳与用途后缀的备份，配一键还原脚本；备份超过 7 天自动清。
- 验证只看真实结果：前端改动用无头浏览器看真实 DOM；投递成功查投递结果而不是任务状态；公网可达性用外部探测；「拆干净了没」要同时查进程、公钥、内存盘、计划任务四样。
- 最小暴露：后端只绑回环；公网只白名单接入必需的几条接口；带密码/密钥的接口只留在私网站点。
- 最小权限：接入服务用独立系统账户，读不到用户工作区；隧道用户受限、只在内存盘认钥匙。
- 确认门：远程执行 default-deny，写与危险必须先批准。
- 不变量：基础设施改动先问主人；夜间只做批准清单；未完成待办不自动顺延。

十八、已知限制与技术债

1. 两个枢纽文件：`todo_api.py`（915 行）与 `remote-api.py`（718 行）承担了太多职责，改动风险高，只能由管家动。
2. 手机语音：安卓浏览器的语音识别要走境外服务，国内不通；要修得把转写搬到自家服务器（未开工）。
3. 内存：4 GB 常年在边缘，重活必须外派；再往上加功能前应先扩内存或迁服务。
4. 浏览器自动化：三号、四号的通道依赖页面结构与登录态（后者约 30 天要重扫一次）。
5. `m6.html` 是生成物：改 `m.html` 必须重跑生成脚本，否则两边漂移。
6. 阶段边界常量两处同步：规划页与出图脚本各一份，容易漏。
7. 外部服务额度：三号每答一句消耗它平台一次额度，不适合高频使用。

文档到此。姊妹篇：《咕嘎系统 · 功能文档》（功能视角与优化历程）。